

Облачные технологии и DevOps

обзор двух взаимосвязанных областей
— **облачных технологий** и **DevOps**.
Вместе они образуют современный
фундамент для разработки,
развертывания и эксплуатации
приложений.

Содержание

- **Облачные технологии (Cloud Computing)**
- **Модели обслуживания (Service Models)**
- **Модели развертывания (Deployment Models)**
- **DevOps**
- **Ключевые практики DevOps**
- **Популярные инструменты DevOps**
- **Взаимосвязь облачных технологий и DevOps**
- **CI/CD-инструменты**

1. Облачные технологии (Cloud Computing)

1.1. Определение и суть

Облачные технологии — это модель предоставления вычислительных ресурсов (серверы, хранилища, базы данных, сети, ПО) через интернет по требованию, с оплатой по факту использования.

Пользователь получает доступ к ресурсам без необходимости владеть и обслуживать физическую инфраструктуру.

1.2. Основные характеристики (по NIST

The National Institute of Standards and Technology,)

Самообслуживание по требованию – ресурсы выделяются автоматически, без взаимодействия с провайдером.

Широкий сетевой доступ – доступ возможен из любой точки через стандартные протоколы.

Пулы ресурсов – физические ресурсы объединяются для обслуживания множества клиентов.

Быстрая эластичность – ресурсы масштабируются динамически, вплоть до автоматического.

Измеряемый сервис – использование отслеживается и тарифицируется.

1.3. Модели обслуживания (Service Models)

Модель	Что предоставляется	Управление со стороны клиента	Примеры
IaaS (Infrastructure as a Service)	Виртуальные машины, сети, диски	ОС, middleware, приложения, данные	AWS EC2, Google Compute Engine, Azure Virtual Machines
PaaS (Platform as a Service)	Платформа для разработки (среда выполнения, базы данных)	Приложения и данные	Google App Engine, Heroku, Azure App Service
SaaS (Software as a Service)	Готовое приложение для конечных пользователей	Пользовательские настройки	Google Workspace, Salesforce, Microsoft 365

1.4. Модели развертывания (Deployment Models)

- **Публичное облако** – ресурсы принадлежат провайдеру и предоставляются широкой публике.
- **Частное облако** – инфраструктура используется исключительно одной организацией (может управляться самой организацией или внешним поставщиком).
- **Гибридное облако** – комбинация публичного и частного облака с оркестрацией между ними.
- **Мультиоблако** – использование нескольких публичных облачных провайдеров одновременно

1.5. Ключевые игроки

- **Amazon Web Services (AWS)** – лидер по функциональности и рыночной доле.
- **Microsoft Azure** – сильная интеграция с продуктами Microsoft.
- **Google Cloud Platform (GCP)** – экспертиза в big data, AI/ML.
- Также: Alibaba Cloud, IBM Cloud, Oracle Cloud.

2. DevOps

2.1. Определение и философия

DevOps (Development + Operations) — это набор практик, направленных на объединение разработки (Dev) и эксплуатации (Ops) для ускорения доставки приложений, повышения их надежности и улучшения взаимодействия команд.

DevOps не является инструментом или должностью, это культурный сдвиг и подход к организации процессов.

2.2. Принципы (CALMS)

- **Culture** – культура сотрудничества, общая ответственность за продукт.
- **Automation** – автоматизация повторяющихся задач.
- **Lean** – применение бережливых принципов (уменьшение потерь, сокращение цикла обратной связи).
- **Measurement** – измерение метрик для постоянного улучшения.
- **Sharing** – обмен знаниями и инструментами между командами.

2.3. Ключевые практики DevOps

- **CI/CD (Continuous Integration / Continuous Delivery / Deployment)**
- **CI** – регулярное слияние кода разработчиков в общий репозиторий с автоматической сборкой и тестированием.
- **CD (Delivery)** – автоматическая подготовка артефакта к развертыванию на любом окружении.
- **CD (Deployment)** – автоматическое развертывание изменений в production после успешного прохождения тестов.

Ключевые практики DevOps

Инфраструктура как код (IaC)

Управление инфраструктурой через декларативные конфигурации (Terraform, AWS CloudFormation, Ansible).

Это позволяет версионировать инфраструктуру, переиспользовать конфигурации и автоматически создавать/уничтожать окружения.

Мониторинг и наблюдаемость (Observability)

Сбор метрик, логов и трассировок для понимания состояния системы.

Ключевые столпы наблюдаемости: метрики, логи, трейсы.

Микросервисная архитектура

Разделение приложения на слабо связанные, независимо развертываемые сервисы, что ускоряет поставку и облегчает масштабирование.

Безопасность (DevSecOps)

Интеграция практик безопасности на всех этапах жизненного цикла (shift-left): статический анализ кода, сканирование контейнеров, управление секретами.

2.4. Популярные инструменты DevOps

Категория	Инструменты
Системы контроля версий	Git (GitHub, GitLab, Bitbucket)
CI/CD	Jenkins, GitLab CI, GitHub Actions, CircleCI, Azure Pipelines
IaC	Terraform, Pulumi, AWS CloudFormation, Ansible, Chef, Puppet
Контейнеризация	Docker, containerd
Оркестрация контейнеров	Kubernetes, Docker Swarm, Amazon ECS
Мониторинг и логи	Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana), Datadog
Управление секретами	HashiCorp Vault, AWS Secrets Manager

3. Взаимосвязь облачных технологий и DevOps

Облако и DevOps эволюционировали вместе и сегодня являются взаимодополняющими концепциями:

Аспект	Как облако помогает DevOps
Автоматизация	API облаков позволяют полностью автоматизировать создание инфраструктуры (IaC).
Эластичность	Возможность динамически выделять ресурсы под нужды CI/CD и самих приложений.
Самообслуживание	Разработчики могут самостоятельно создавать окружения через self-service порталы.
Глобальная доступность	Упрощает распределенные команды и развертывание в нескольких регионах.
Платежная модель	Оплата по факту снижает барьеры для экспериментов и частых развертываний.

Взаимосвязь облачных технологий и DevOps

В свою очередь, DevOps практики делают использование облака более эффективным:

- Автоматизированные пайплайны ускоряют доставку приложений в облако.
- IaC обеспечивает воспроизводимость и соответствие требованиям безопасности.
- Наблюдаемость позволяет контролировать затраты и производительность облачных ресурсов.

4. Преимущества и вызовы

4.1. Преимущества

- **Скорость вывода на рынок** – от идеи до production за часы, а не месяцы.
- **Масштабируемость** – системы растут вместе с бизнесом.
- **Надежность** – автоматическое восстановление, развертывания без простоев.
- **Эффективность затрат** – плата только за потребленное; оптимизация ресурсов.
- **Улучшенное взаимодействие команд** – устранение «стены» между разработкой и эксплуатацией.

4.2. Вызовы

- **Культурные изменения** – требуется переход от иерархических структур к кросс-функциональным командам.
- **Безопасность** – необходимо встраивать безопасность в процессы, а не накладывать в конце.
- **Сложность** – большое количество инструментов и технологий требует высокой квалификации.
- **Управление затратами** – неконтролируемое использование облака может привести к непредвиденным расходам.
- **Технический долг** – быстрая поставка может накапливать «костыли», если не следить за качеством.

5. Тренды и будущее

- **Serverless** – модель, где облачный провайдер полностью управляет серверами, а разработчик пишет только код (AWS Lambda, Azure Functions). Еще больше упрощает эксплуатацию.
- **GitOps** – управление инфраструктурой и приложениями через Git как единый источник истины.
- **AIOps / MLOps** – применение искусственного интеллекта для анализа операционных данных и автоматизации реагирования на инциденты, а также практики для поставки ML-моделей.
- **Платформенная инженерия** – создание внутренних платформ (IDP), которые скрывают сложность облака и DevOps от разработчиков, предоставляя унифицированный интерфейс.
- **Экологичность** – повышение энергоэффективности облачных ЦОД и практик Green DevOps.

Заключение

Облачные технологии и DevOps образуют современный стандарт для создания и эксплуатации программного обеспечения. Облако предоставляет гибкую, программируемую инфраструктуру, а DevOps — культуру и инструменты для максимально быстрой и надежной доставки ценности пользователям. Вместе они позволяют организациям ускорить инновации, повысить устойчивость систем и адаптироваться к изменяющимся условиям рынка.

1. Что такое CI/CD-инструменты?

CI/CD (Continuous Integration / Continuous Delivery) – это практика автоматизации сборки, тестирования и развертывания приложений. Инструменты CI/CD позволяют:

Автоматически запускать сборку и тесты при каждом изменении кода.

Формировать развёртываемые артефакты.

Выполнять деплой на различные окружения (staging, production).

Обеспечивать качество кода с помощью статического анализа, проверки безопасности и т.д.

Выбор инструмента часто зависит от существующей инфраструктуры, требований к масштабируемости и предпочтений команды.

2. Jenkins

2.1. Общая характеристика

Jenkins – старейший и наиболее распространённый open-source CI/CD-инструмент (изначально ответвление от Hudson, 2005 год). Написан на Java, распространяется под лицензией MIT.

Архитектура: master + агенты (nodes). Master управляет планированием заданий и хранит конфигурации, агенты выполняют задачи на разных платформах (Linux, Windows, macOS).

Гибкость: практически неограниченные возможности благодаря огромной экосистеме плагинов (более 1800).

Способы конфигурации:

Веб-интерфейс с ручным созданием заданий (freestyle jobs).

Pipeline as Code – сценарии на Groovy (Declarative или Scripted pipeline), хранящиеся в Jenkinsfile вместе с кодом.

2.2. Ключевые особенности

- **Плагины** – основная сила Jenkins. Интеграции со всем: Git, Docker, Kubernetes, AWS, Azure, Slack, Jira и т.д. Но из-за обилия плагинов возможны конфликты версий.
- **Масштабируемость** – легко добавлять агенты; облачные провайдеры (AWS, GCP) позволяют динамически запускать агенты для экономии ресурсов.
- **Безопасность**: базовая ролевая модель (матрица доступа), интеграция с LDAP/AD, но настройка может быть сложной.
- **Pipeline** – мощный язык описания процессов, поддерживает параллельные шаги, ручной ввод, условные ветвления.

2.3. Преимущества и недостатки

Преимущества

Бесплатный, open-source, огромное сообщество

Максимальная гибкость и расширяемость

Работает в любом окружении (on-prem, cloud, hybrid)

Огромный выбор плагинов

Поддержка любых языков и стеков

Недостатки

Требует самостоятельной установки и обслуживания

Интерфейс устарел, навигация может быть запутанной

Сложная настройка прав доступа и резервного копирования

При большом количестве плагинов возможна нестабильность

Pipeline на Groovy требует изучения DSL

2.4. Когда выбирать Jenkins

У вас уже есть сложная on-prem инфраструктура.

(**On-prem** (сокращение от on-premise) —

модель развёртывания программного обеспечения и ИТ-систем, при которой все ресурсы (серверы, данные, ПО) размещаются и управляются **на собственной инфраструктуре компании**, а не в облаке стороннего провайдера)

Требуется глубокая кастомизация и интеграция с унаследованными системами.

Команда готова тратить время на

администрирование и поддержку сервера.

Нужна возможность работы без доступа в интернет.

3. GitLab CI

3.1. Общая характеристика

- **GitLab CI** – встроенный CI/CD-движок платформы GitLab. Появился как самостоятельный продукт (GitLab CI), а затем был полностью интегрирован в единый GitLab-сервер. Является open-core – бесплатная версия имеет широкие возможности, а дополнительные фичи доступны в платных тарифах.
- **Архитектура:** центральный GitLab-сервер + **раннеры** (runners). Раннеры – агенты, которые выполняют задачи. Могут быть установлены на виртуальных машинах, в Kubernetes, или использоваться облачные раннеры от GitLab.
- **Конфигурация:** единственный файл .gitlab-ci.yml в корне репозитория. Описывает стадии (stages), задания (jobs), переменные окружения, артефакты и т.д.
- **Интеграция:** всё в одном – репозиторий, issue tracker, CI/CD, реестр контейнеров, реестр пакетов.

3.2. Ключевые особенности

- **Автоматическое обнаружение** – как только в репозитории появляется `.gitlab-ci.yml`, GitLab автоматически запускает пайплайны при `push` или `merge request`.
- **Гибкие стадии** – можно определить произвольные стадии (`build`, `test`, `deploy`) и управлять порядком их выполнения.
- **Среда и развертывание** – встроенная поддержка окружений (`environments`), защищённых переменных, ручных действий (`when: manual`).
- **Auto DevOps** – автоматическая настройка пайплайна для типовых приложений (нужно только включить опцию).
- **Безопасность** – статический анализ (SAST), сканирование зависимостей, управление секретами (в платных версиях).

3.3. Преимущества и недостатки

Преимущества

Тесная интеграция
репозитория и CI/CD

Один файл
конфигурации, простота
входа

Встроенные реестры
(container, package)

Удобный веб-интерфейс с
визуализацией
пайплайнов

Поддержка Kubernetes-
раннеров и
автомасштабирования

Недостатки

Требует хостинга GitLab
(self-managed или cloud)

Более сложные сценарии
могут приводить к
громоздкому .gitlab-ci.yml

В бесплатной версии
ограничено количество
раннеров (при
использовании облачных
раннеров)

Отсутствует такая же
гибкость плагинов, как у
Jenkins

Некоторые продвинутые
возможности доступны
только в платных
тарифах

3.4. Когда выбирать GitLab CI

- Вы уже используете GitLab как систему контроля версий.
- Нужна единая платформа «всё в одном» (репозиторий, CI/CD, реестры).
- Команда предпочитает YAML-конфигурацию и не хочет настраивать отдельный CI-сервер.
- Вы хотите быстро начать с минимальными настройками.

4. GitHub Actions

4.1. Общая характеристика

- **GitHub Actions** – CI/CD-платформа, встроенная в GitHub. Анонсирована в 2018 году, быстро стала популярной благодаря тесной интеграции с экосистемой GitHub и удобному синтаксису.
- **Архитектура:** события GitHub (push, pull request, issue, schedule) запускают **воркфлоу** (workflows), которые описываются в YAML-файлах в директории `.github/workflows/`. Выполнение происходит на **раннерах** – либо предоставленных GitHub (бесплатные ограниченные ресурсы), либо self-hosted.
- **Конфигурация:** YAML-файлы с определением заданий (jobs) и шагов (steps). Каждый шаг может быть shell-командой или использовать **action** – переиспользуемый компонент из маркетплейса.
- **Экосистема:** GitHub Marketplace содержит тысячи готовых actions (например, checkout, setup-node, deploy-to-aws).

4.2. Ключевые особенности

- **События** – гибкая настройка триггеров: push, pull_request, schedule, workflow_dispatch (ручной запуск), а также события от внешних систем (repository_dispatch).
- **Матричные сборки** – встроенная поддержка запуска одного задания на нескольких версиях языка, ОС и т.д. без дублирования кода.
- **Среда выполнения** – можно смешивать шаги на разных ОС (ubuntu, windows, macos) в рамках одного воркфлоу.
- **Кэширование** – встроенный механизм кэширования зависимостей для ускорения сборок.
- **Безопасность** – поддержка окружений (environments) с защитой, секреты, встроенная система проверок для pull request.

4.3. Преимущества и недостатки

Преимущества

Полная интеграция с GitHub (репозитории, PR, issues)

Простой YAML-синтаксис, низкий порог входа

Огромный маркетплейс готовых action-ов

Встроенная поддержка контейнеров и оркестрации

Self-hosted раннеры для полного контроля

Недостатки

Привязанность к GitHub – нельзя использовать без GitHub

Для сложных сценариев может не хватать возможностей (хотя action-ы помогают)

Бесплатный лимит для публичных репозиторий, для частных – зависит от тарифа

Меньше возможностей для сложных разветвленных пайплайнов по сравнению с Jenkins Pipeline

Логи сборки хранятся ограниченное время (в облачной версии)

4.4. Когда выбирать GitHub Actions

- Ваш код хранится на GitHub (особенно если это открытые проекты или организация использует GitHub Enterprise).
- Нужно быстро настроить CI/CD без развёртывания дополнительной инфраструктуры.
- Вы хотите использовать готовые компоненты из маркетплейса для ускорения разработки.
- Команда уже привыкла к GitHub flow и хочет единого интерфейса.

Сравнение

Характеристика	Jenkins	GitLab CI	GitHub Actions
Тип	Open-source, self-hosted	Open-source core, self-hosted или SaaS	SaaS (в составе GitHub), self-hosted раннеры
Конфигурация	Jenkinsfile (Groovy) или UI	.gitlab-ci.yml	.github/workflows/*.yaml
Язык описания	Groovy (Declarative/Scripted)	YAML	YAML + actions (JS/Docker)
Агенты	Master + агенты (подключаются)	Раннеры (подключаются к GitLab)	Раннеры (GitHub-hosted или self-hosted)
Экосистема	Плагины (>1800)	Встроенные возможности + интеграции через API	GitHub Marketplace (actions)
Сложность установки	Высокая (требует настройки)	Средняя (GitLab сам управляет CI)	Низкая (в облаке) или средняя (self-hosted)
Масштабируемость	Высокая (динамические агенты)	Высокая (Kubernetes-раннеры)	Высокая (self-hosted раннеры в K8s)
Визуализация	Плагины (Blue Ocean)	Встроенная (граф пайплайнов)	Встроенная (граф вкладки Actions)
	Бесплатно (только для self-hosted)	Бесплатно для self-hosted	Бесплатно для публичных репозиторий

6. Как выбрать инструмент?

6.1. Jenkins

Подойдёт, если:

Нужна максимальная гибкость и поддержка legacy-систем.

Уже есть инфраструктура для его размещения.

Команда готова заниматься администрированием.

Не подойдёт, если:

Хочется быстрого старта без настройки сервера.

Отсутствует опыт управления Java-приложениями и плагинами.

6.2. GitLab CI

Подойдёт, если:

- GitLab уже используется как VCS.
- Нужна единая платформа с репозиторием, CI/CD и реестрами.
- Предпочитаете конфигурацию через YAML и не хотите возиться с Jenkins.

Не подойдёт, если:

- Вы не используете GitLab и не планируете переходить.
- Нужны специфические плагины, которых нет в экосистеме GitLab.

6.3. GitHub Actions

Подойдёт, если:

Репозитории уже на GitHub (особенно публичные).
Нужно быстро настроить CI/CD с минимальными усилиями.

Хотите использовать готовые действия (actions) из маркетплейса.

Не подойдёт, если:

Вы не используете GitHub или ваша организация не готова переходить на него.

Требуется on-prem решение без доступа к интернету.

Заключение

Все три инструмента активно развиваются и способны покрыть потребности большинства проектов:

Jenkins остаётся выбором для сложных, highly-customizable пайплайнов и организаций, которые предпочитают полный контроль над инфраструктурой.

GitLab CI предлагает сбалансированное решение с глубокой интеграцией, удобным UI и возможностью self-hosted.

GitHub Actions лидирует в простоте использования и скорости настройки, особенно для open-source и команд, уже работающих в экосистеме GitHub.

Выбор конкретного инструмента зависит от текущего технологического стека, доступных ресурсов на поддержку и предпочтений команды.

В современных реалиях также часто встречаются гибридные сценарии (например, GitLab для кода, а Jenkins для специфических задач).

Литература

- [https://ru.wikipedia.org/wiki/Amazon Web Services](https://ru.wikipedia.org/wiki/Amazon_Web_Services)
- <https://practicum.yandex.ru/blog/kto-takoy-devops-inzhener/>
- <https://habr.com/ru/articles/862766/>